

Shell Scripting: Loops, Arguments & Error Handling

❖ Task 1: create `for_loop.sh` that:

- Loops through a list of 5 fruits and prints each one
 - `#!/bin/bash`
 - `fruits=("apples" "banana" "grapes" "mango" "oranges")`
 - `for fruit in "${fruits[@]}"`
 - `do`
 - `echo "$fruit"`
 - `done`

```
ubuntu@ip-172-31-47-149:~$ ./for_loop.sh
apples
banana
grapes
mango
oranges
```

- Create `count.sh` that:
- Prints numbers 1 to 10 using a for loop
 - `#!/bin/bash`
 - `for (( num=1 ;num<=10; num++ ))`
 - `do`
 - `echo "$num"`
 - `Done`

```
ubuntu@ip-172-31-47-149:~$ vim count.sh
ubuntu@ip-172-31-47-149:~$ chmod 764 count.sh
ubuntu@ip-172-31-47-149:~$ ./count.sh
1
2
3
4
5
6
7
8
9
10
```

❖ Task 2: While Loop

- Create countdown.sh that:
- Takes a number from the user
- Counts down to 0 using a while loop
- Print “Done!” at the end
 - Read -p “Enter the number:” num
 - While [\$num -ge 0]
 - Do
 - Echo “\$num”
 - Num=\$((num-1))
 - Sleep 1
 - Done
 - Echo “Done!”

```
ubuntu@ip-172-31-47-149:~$ vim countdown.sh
ubuntu@ip-172-31-47-149:~$ ./countdown.sh
Enter the number:15
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
0
Done!
```

❖ Task 3: Command-Line Arguments

- Create greets.sh that:
- Accepts a name as \$1

- Prints Hello, <names>!
- If no argument is passed, prints “Usage: ./greet.sh”
 - #!/bin/bash
 - If [-z “\$1”]; then
 - Echo “Usage: ./greet.sh <name>”
 - Exit 1
 - Fi
 - Echo “Hello, \$1!”

```
ubuntu@ip-172-31-47-149:~$ vim greet.sh
ubuntu@ip-172-31-47-149:~$ ./greet.sh
Usage: ./greet.sh <name>
ubuntu@ip-172-31-47-149:~$ ./greet.sh Ameeth
Hello, Ameeth!
ubuntu@ip-172-31-47-149:~$ |
```

❖ Create args_demo.sh

- Print total number of arguments (\$#)
- Prints all arguments (@\$)
- Prints all the scripts (\$0)
 - #!/bin/bash
 - Echo “Total number of arguments: \$#”
 - Echo “All arguments: @\$”
 - Echo “Script name: \$0”

```
ubuntu@ip-172-31-47-149:~$ vim args_demo.sh
ubuntu@ip-172-31-47-149:~$ chmod 764 args_demo.sh
ubuntu@ip-172-31-47-149:~$ ls -l args_demo.sh
-rwxrwx-r-- 1 ubuntu ubuntu 203 Feb 18 08:58 args_demo.sh
ubuntu@ip-172-31-47-149:~$ ./args_demo.sh
Total number of arguments: 0
All arguments:
Script name: ./args_demo.sh
ubuntu@ip-172-31-47-149:~$ ./args_demo.sh hello this is rohan, today im
learning shell scripting
Total number of arguments: 9
All arguments: hello this is rohan, today im learning shell scripting
Script name: ./args_demo.sh
```

❖ Task 4: Install packages via Scripts

- Create install_packages.sh that:

- Define a list of packages: nginx, curl, wget
- Loop through the list
- Checks if each package is installed (use dpkg -s or rpm - q)
- Installs it if missing, skips if already present
- Prints status for each package
 - Packages=("nginx" "curl" "wget")
 - For i in "\${packages[@]}";
 - Do
 - If dpkg -s \$i &> /dev/null; then
 - Else
 - Echo "Installing \$i...."
 - Apt-get update -qq
 - Apt-get install -y \$i && echo "\$i installed successfully."

```

root@ip-172-31-47-149:/# vim install_packages.sh
root@ip-172-31-47-149:/# chmod 764 install_packages.sh
root@ip-172-31-47-149:/# ./install_packages.sh
Installing nginx....
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 53 not upgraded.
Need to get 565 kB of archives.
After this operation, 1596 kB of additional disk space will be used.
Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd6
4 nginx-common all 1.24.0-2ubuntu7.6 [43.5 kB]
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd6
4 nginx amd64 1.24.0-2ubuntu7.6 [521 kB]
Fetched 565 kB in 0s (25.9 MB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 72190 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.24.0-2ubuntu7.6_all.deb ...
Unpacking nginx-common (1.24.0-2ubuntu7.6) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.24.0-2ubuntu7.6_amd64.deb ...
Unpacking nginx (1.24.0-2ubuntu7.6) ...
Setting up nginx-common (1.24.0-2ubuntu7.6) ...
Setting up nginx (1.24.0-2ubuntu7.6) ...
  * Upgrading binary nginx [ OK ]
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for ufw (0.36.2-6) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

```

❖ Task 5: Error Handling

- Create safe_script.sh that:
 - Uses set -e at the top (exit on error)
 - Tries to create a directory /tmp/devops_test
 - Tries to navigate into it
 - Creates a file inside
 - Uses || operator to print an error if any step fails.
 - Mkdir -p /tmp/devops_test || { echo "Error: Failed to create /tmp/devops_test"; exit 1; }
 - Cd /tmp/devops_test || { echo "Error: Failed to create /tmp/devops_test"; exit 1; }
 - Touch testfile.txt || echo "Error: Failed to create testfile.txt"

- Echo “Success: All operations completed successfully.”

```
ubuntu@ip-172-31-47-149:~$ vim safe_script.sh
ubuntu@ip-172-31-47-149:~$ chmod 764 safe_script.sh
ubuntu@ip-172-31-47-149:~$ ./safe_script.sh
Success: All operations completed successfully.
ubuntu@ip-172-31-47-149:~$ |
```

❖ What I have learned (3 key points)

- If any command fails, exit the script immediately after the “set -e” is used at the beginning of the script.
- To create scripts that check for package installation status.
- Loops: for and while loops make repetitive tasks simple and readable.